



Retrofit

Laboratorio de Software 2025



¿Qué es Retrofit?

Retrofit es una librería para Android y Java, compatible con Kotlin, desarrollada por Square, la cual nos permite realizar peticiones http de forma simple.

Característica principal:

→ Facilidad en la interacción con las API

Retrofit 2 Converter Gson: Herramienta encargada de convertir un JSON a una Data Class.

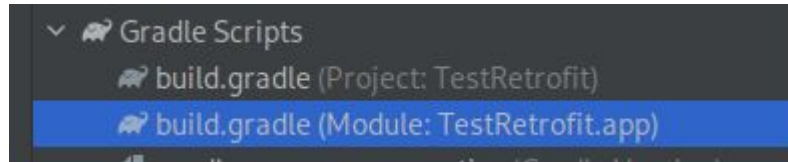
Manifest

En el archivo AndroidManifest.xml se deben incluir los permisos para que la app pueda acceder a internet.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Dependencias

```
dependencies {  
  
...  
  
implementation(libs.retrofit)  
  
implementation(libs.retrofit.converter.gson)  
  
implementation(libs.gson)  
  
}
```



Modelos

```
data class User(  
    val id: Int?,  
    val email: String?,  
    val password: String?,  
    val firstName: String?,  
    val lastName: String?,  
    val avatar: String?  
) {
```

```
data class UsersPage(  
    val page: Long,  
    val perPage: Long,  
    val total: Long,  
    val totalPages: Long,  
    val data: List<User>,  
    val support: Support  
) {
```

Request

/api/users?page=2

Response

200

```
{  
  "page": 2,  
  "per_page": 6,  
  "total": 12,  
  "total_pages": 2,  
  "data": [  
    {  
      "id": 7,  
      "email": "michael.lawson@reqres.in",  
      "first_name": "Michael",  
      "last_name": "Lawson",  
      "avatar": "https://reqres.in/img/faces/  
    },  
  ],  
}
```

Retrofit

```
val api = Retrofit.Builder().baseUrl("https://reqres.in/")  
    .addConverterFactory(GsonConverterFactory.create())  
    .build().create(UserService::class.java)
```

```
interface UserService {  
  
    @GET("api/users") // api/users?page=1  
    suspend fun getUsers(@Query("page") page: Long): Response<UsersPage>  
  
    @GET("api/users/{id}")  
    suspend fun getSingleUser(@Path("id") id: Long): Response<SingleUser>  
  
    @POST("/api/login")  
    suspend fun login(@Body user: User): Response<Map<String, String>>  
  
    @GET("/api/users")  
    suspend fun getUsersWithAuth(@Header("authorization") token: String): Response<User>  
}
```

Ejemplo GET

```
interface UserService {  
    ...  
    @GET("api/users") // api/users?page=1  
    suspend fun getUsers(@Query("page") page: Long): Response<UsersPage>  
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    ...  
    lifecycleScope.launch {  
        val result = api.getUsers(1)  
        if(result.isSuccessful) {  
            println("Codigo ${result.code()}")  
            println(result.body())  
        } else{  
            println("Codigo ${result.code()}")  
            println(result.errorBody())  
        }  
    }  
}
```

Ejemplo GET

```
interface UserService {  
    ...  
    @GET("api/users/{id}")  
    suspend fun getSingleUser(@Path("id") id: Long): Response<SingleUser>  
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    ...  
    lifecycleScope.launch {  
        val result = api.getSingleUser(1)  
        if(result.isSuccessful) {  
            println("Codigo ${result.code()}")  
            println(result.body())  
        } else{  
            println("Codigo ${result.code()}")  
            println(result.errorBody())  
        }  
    }  
}
```

Ejemplo POST

```
interface UserService {  
    ...  
    @POST("/api/login")  
    suspend fun login(@Body user: User): Response<Map<String, String>>  
}
```

```
{  
    "token": "QpwL5tke4Pnpja7X4"  
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    ...  
    lifecycleScope.launch {  
        val user = User("sebastian@laboratorio.com", "12345678", )  
        val result = api.login(user)  
        if(result.isSuccessful) {  
            println("Codigo ${result.code()}")  
            println(result.body()?.get("token"))  
        } else{  
            println("Codigo ${result.code()}")  
            println(result.errorBody())  
        }  
    }  
}
```

Agregar parámetros al HEADER

```
interface UserService {  
    ...  
    @GET("api/users/{id}")  
    suspend fun getUsersWithAuth(@Header("authorization") token: String): Response<UsersPage>  
}
```

```
override fun onCreate(savedInstanceState: Bundle?) {  
    ...  
    lifecycleScope.launch {  
        val result = api.getUsersWithAuth(token)  
        if(result.isSuccessful) {  
            println("Codigo ${result.code()}")  
            println(result.body())  
        } else{  
            println("Codigo ${result.code()}")  
            println(result.errorBody())  
        }  
    }  
}
```

Interceptores

Los interceptores son componentes que permiten interceptar y modificar tanto las solicitudes salientes como las respuestas que llegan a la app. Con Retrofit, estos interceptores nos permiten aplicar lógica común a todas las peticiones, como agregar encabezados, registrar información o manejar errores.

En nuestro caso, utilizaremos un interceptor para inyectar automáticamente el token de autorización en cada request.

Interceptores

```
class AuthInterceptor : Interceptor {  
  
    override fun intercept(chain: Interceptor.Chain): Response {  
  
        val originalRequest = chain.request()  
  
        val requestBuilder = originalRequest.newBuilder()  
  
        requestBuilder.header("Authorization", TOKEN)  
  
        return chain.proceed(requestBuilder.build())  
  
    }  
  
}
```

Interceptores

```
val authInterceptor = AuthInterceptor()

val okHttpClient = OkHttpClient.Builder()
    .addInterceptor(authInterceptor)
    .build()

val retrofit = Retrofit.Builder()
    .baseUrl("https://reqres.in/") .client(okHttpClient)
    .addConverterFactory(GsonConverterFactory.create())
    .build()

val api = retrofit.create(UserService::class.java)
```

Docs

<https://developer.android.com/develop/connectivity/network-ops/connecting>

<https://square.github.io/retrofit/>

<https://square.github.io/okhttp/features/interceptors/>